

ARDUINO BASED DIGITAL CLOCK WITH ALARM

August 28, 2026

Team Members

- V. Keerthana - EE26BTECH11232
- P. Anvidha Reddy - EE26BTECH11221
- Sushmitha Ankireddy - EE26BTECH11229

1 Introduction

This project represents our very first engineering project in college, undertaken as a team of three. The primary objective of this project was to design and construct a functional digital clock using an Arduino microcontroller platform.

Digital clocks are electronic systems that represent time numerically rather than using traditional clock hands. In this project, a digital clock was designed using an Arduino as the central controller and 7-segment displays for displaying the time.

Moving from textbook theory to actual breadboards proved to be a steep learning curve.

More than just a technical assignment, it became a practical, hands-on lesson in real-world learning, failure, and patience.

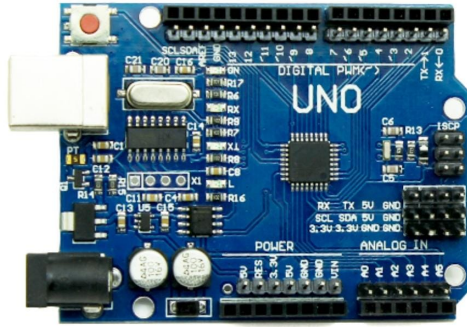
2 Objectives

- To design and build a functional digital clock controlled by an Arduino and push buttons.
- To interface 7-segment displays using 7447 decoder ICs for digital time display.
- To integrate a buzzer for a working alarm function.
- To gain practical experience in hardware assembly, microcontroller programming, and circuit debugging.

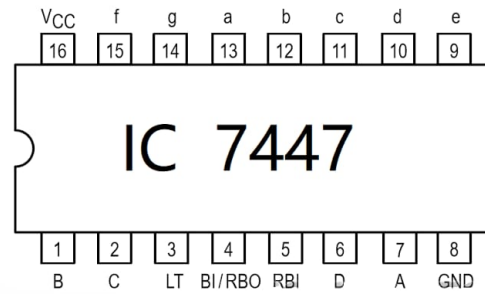
3 Components Used

To construct the digital clock, the following components were utilized:

Component	Quantity	Usage
Arduino Microcontroller	1	Controls time logic and sending signals.
7447 BCD Decoder IC	1	Converts binary code to display numbers.
7-Segment Display (Common Anode)	6	Displays numbers for hours, minutes, seconds.
Push Buttons	4	Adjusts hours and minutes manually.
Breadboards	2	Holds circuits and components together.
Jumper Wires	~60	Connects power and signal paths.
Buzzer	1	Converts electrical signals into loud sound.
Resistors	7	to control current and protect displays.



(a) Arduino Uno



(b) 7447

Arduino Uno R3 Parts and Functions:

- **ATmega328P Microcontroller:** The main processor that stores and executes the project code.
- **Digital I/O Pins (0–13):** Sends and receives digital HIGH/LOW signals.
- **Analog Input Pins (A0–A5):** Reads variable voltage signals from sensors.
- **Power & Ground Pins (5V, 3.3V, GND):** Supplies power and ground reference to components.

7447 BCD Decoder IC:

- **Purpose:** Converts 4-bit binary code (BCD) from the Arduino into numbers (0–9) on a 7-segment display.
- **Power Pins:** Connects to +5V (Pin 16) and Ground (Pin 8).
- **Control Pins (3, 4, 5):** Tied to +5V for standard operation; used for segment testing and display blanking.

3.1 Testing Gone Wrong

Before assembling the full circuit, we tested the 7-segment display by connecting it directly to the Arduino's 5V pin.

However, we accidentally forgot to include a current-limiting resistor. The resulting excess current caused a short circuit, instantly blowing up the display.

This mistake taught us the critical importance of always using resistors to protect sensitive components from overcurrent.

3.2 Correction and Successful Working

We added current-limiting resistors inline with the display lines to safely limit the current. The 7-segment display lit up correctly and safely displayed the output without any issues.

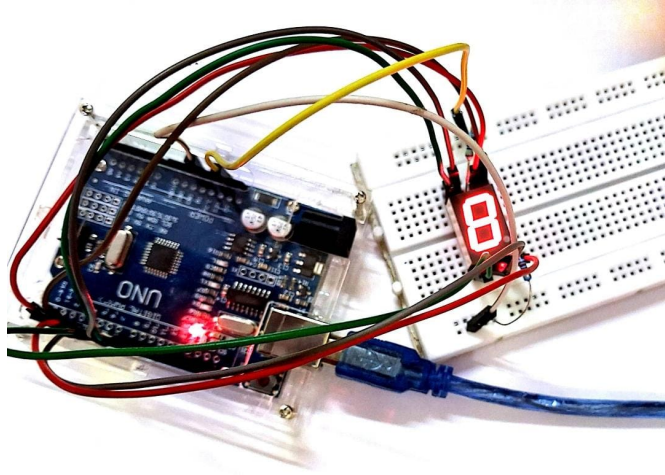


Figure 1: Successful working of the 7-segment display with current-limiting resistors.

4 Hardware Design

1. 7447 IC Inputs:

- Connect inputs A, B, C, D to Arduino pins 2, 3, 4, 5.
- Connect Pin 16 (V_{CC}) to 5V rail and Pin 8 to GND.

2. 7447 IC Outputs:

- Connect output pins 9 to 15 to the display segments *a* through *g* using jumper wires and resistors. We used Daisy chaining, its a hardware connection method where multiple components or devices are wired together sequentially in a series—passing power, data, or signals from one unit directly to the next instead of running separate connections from each device back to a central controller.

3. 7-Segment Display Common Pins:

- Connect the middle pins (common anode) of the displays to Arduino pins 6,7,8,9,10,11.

4. Push Buttons:

- Connect 1st push button (2 legs) to Arduino pin A2 and GND which resets hours.
- Connect 2nd push button to Arduino pin A3 and GND which resets minutes.
- Connect 3rd push button to Arduino pin A1 and GND which acts as mode switch button.
- Connect 4th push button to Arduino pin A4 and GND which turns alarm ON/OFF or turns off the buzzer.

5. Buzzer:

- Connect positive pin (+) to Arduino pin A0 .
- Connect negative pin (-) to GND.

5 Challenges Encountered and Solutions

5.1 Power Distribution Issue and Fix

During testing, we initially connected the current-limiting resistors directly to the middle pins (common anodes) of each display linked to Arduino pins 8 through 13. This setup led to uneven power distribution, causing the display to appear very dim and flicker inconsistently mainly the d segment of every display.

Table 1: 7447 IC Pinout and Connections

Pin	Name	Connect To	Function
1	Input B	Arduino Pin 3	BCD Bit 1
2	Input C	Arduino Pin 4	BCD Bit 2
3	LT	5V Rail	Lamp test (keep HIGH)
4	BI/RBO	5V Rail	Blanking control (keep HIGH)
5	RBI	5V Rail	Zero blanking (keep HIGH)
6	Input D	Arduino Pin 5	BCD Bit 3 (MSB)
7	Input A	Arduino Pin 2	BCD Bit 0 (LSB)
8	GND	GND Rail	Ground
9	Output e	Segment e ($240\ \Omega$)	Bottom-left segment
10	Output d	Segment d ($240\ \Omega$)	Bottom segment
11	Output c	Segment c ($240\ \Omega$)	Bottom-right segment
12	Output b	Segment b ($240\ \Omega$)	Top-right segment
13	Output a	Segment a ($240\ \Omega$)	Top segment
14	NC	Unused	No connection
15	Output f	Segment f ($240\ \Omega$)	Top-left segment
16	VCC	5V Rail	Power supply

To rectify this, we removed the resistors from the common anode pins and instead placed individual current-limiting resistors on the segment lines (pins 9 to 15 of the 7447 IC). This ensured uniform brightness and stable power across all display segments.

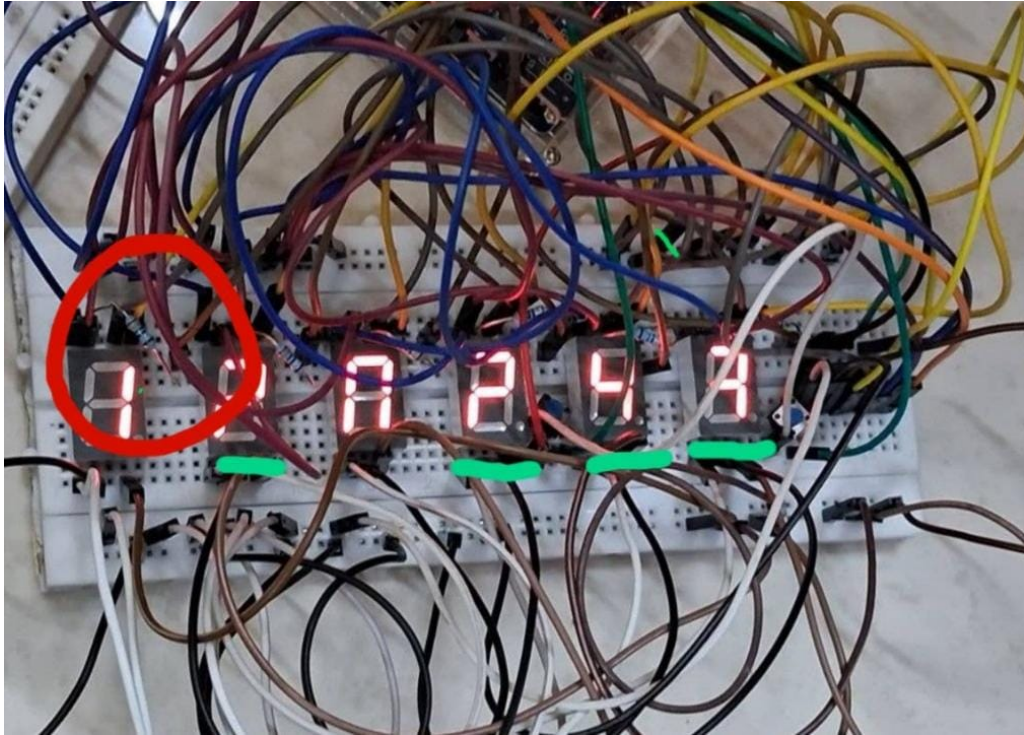


Figure 2: Dim displays and malfunctioning of d segment due to incorrect resistor setup.

5.2 Breadboard Rail Faults and Pin Misplacement

- **IC Relocation Failure:** So while setting up using daisy chaining, segment **b** failed to glow when driven by the 7447 decoder IC. Relocating the IC on the breadboard fixed segment **b**, but segment **g** then stopped working due to faulty breadboard rails or damaged IC outputs.
- **Direct Bypass & Wiring Mistake:** To bypass the fault, segment **g** was connected directly to

the Arduino. It initially stayed ON continuously because it was accidentally plugged into Pin 13 instead of the intended target pin.

- **Final Resolution:** Rehousing segment **g** to Pin 12 resolved the issue, allowing proper active-LOW control, voltage compatibility, and safe current handling directly from the microcontroller.

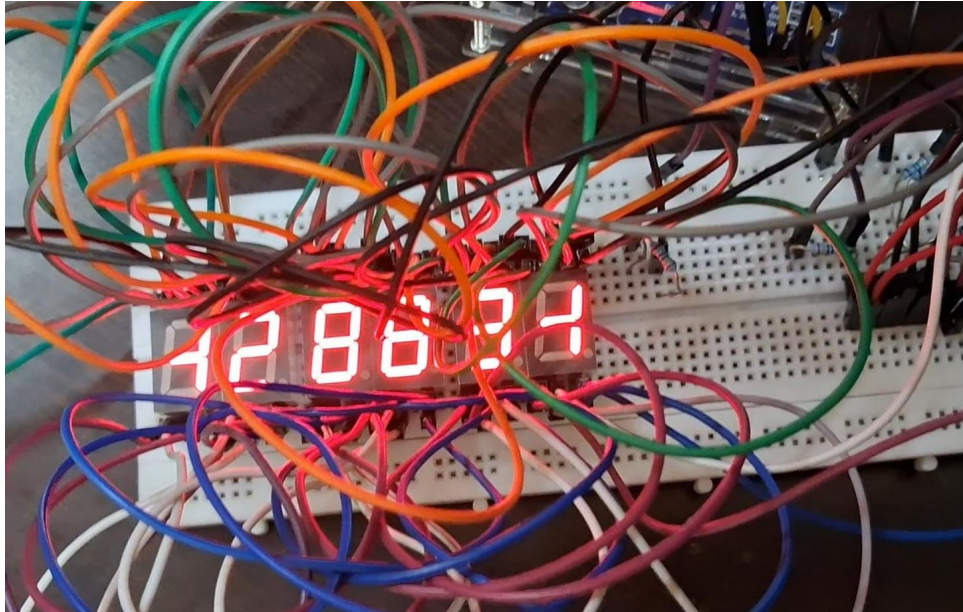


Figure 3: g segment glowing in every display

[\[Click Here to Watch the problem Video\]](#)

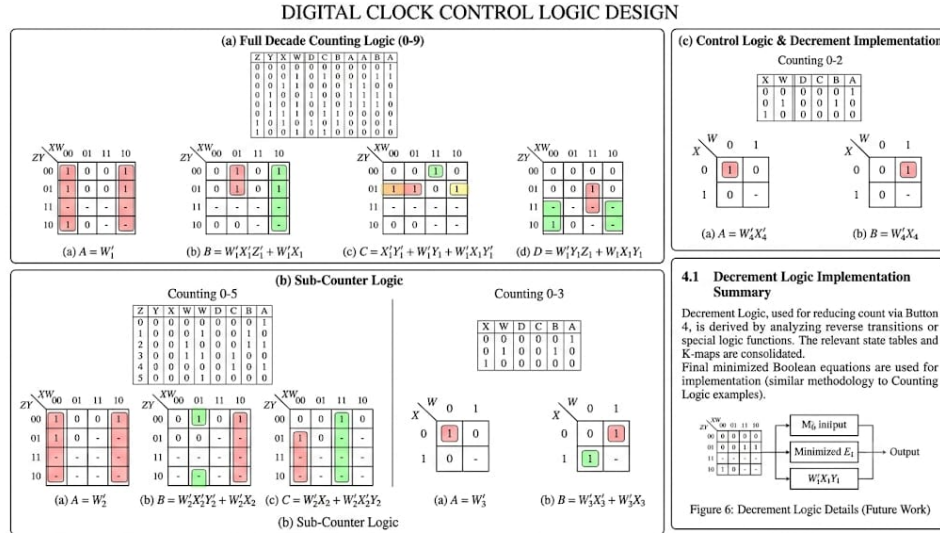
Core Working Principles

- **Persistence of Vision (POV):** Human eyes retain an image for about $\frac{1}{16}$ th of a second. High-speed sequential switching tricks the human eye into seeing all 6 display digits illuminated simultaneously.
- **Hardware Multiplexing:** Cycles power through individual display digits sequentially using Arduino pins 8–13. Sharing 7 segment data lines across 6 physical displays reduces required Arduino pins from 42 down to just 10 total pins.
- **Active-Low BCD Decoding:** The 7447 IC converts 4-bit Binary Coded Decimal inputs (0 to 9) into low-voltage (0V/GND) outputs on segments *a* through *g*, matching the common-anode display setup.
- **Internal Timebase Tracking:** Arduino utilizes its internal hardware timer and crystal oscillator to calculate seconds, minutes, and hours accurately in RAM.

Background Concepts Explained

- **Karnaugh Maps (K-Maps):**
 - A graphical optimization method used to simplify complex Boolean algebra expressions.

- K-map equations are implemented directly inside the logic gates of the 7447 IC to map 4 binary input lines (A, B, C, D) to 7 segment output pins efficiently.
- The Core Mechanism: K-maps convert a truth table into a grid labeled with Gray Code ($00 \rightarrow 01 \rightarrow 11 \rightarrow 10$). Because adjacent cells only change by one bit, visual grouping automatically cancels out changing variables.
- For n input variables, the map contains 2^n cells.
- Grouping (Looping): You group adjacent 1s (for Sum of Products or SOP) or 0s (for Product of Sums or POS) in powers of two (1, 2, 4, 8) to simplify the equation.



4.1 Decrement Logic Implementation Summary

Decrement Logic, used for reducing count via Button 4, is derived by analyzing reverse transitions or special logic functions. The relevant state tables and K-maps are consolidated. Final minimized Boolean equations are used for implementation (similar methodology to Counting Logic examples).

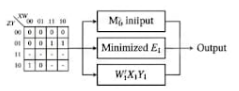


Figure 6: Decrement Logic Details (Future Work)

Figure 4: K Maps

• Button Contact Debouncing:

- Mechanical push buttons generate electrical noise (bouncing) for a few milliseconds upon being pressed.
- Software debouncing code pauses briefly before verifying an input, ensuring a single physical click isn't registered as multiple false triggers.

• Finite State Machine (FSM):

- A code architecture where the program remains locked in one defined execution state at any given moment.
- Governs clean transitions when pressing the Mode button (pin A0) to switch operational modes (e.g., `SHOW_TIME` $\rightarrow$ `SET_HOURS` $\rightarrow$ `SET_MINUTES`).

6 Arduino Source Code Implementation

The following C++ program controls the 6-digit display and alarm system. It executes state transitions using minimized Karnaugh-map (K-Map) Boolean logic routines and outputs BCD signals alongside direct active-LOW segment overrides.

```

1 // =====
2 // 6-Digit Clock & Alarm System - Pure K-Map Engine (12-Hour Format)
3 // =====
4
5 const int pinA = 2; // IC 7447 Pin 7
6 const int pinB = 3; // IC 7447 Pin 1
7 const int pinC = 4; // IC 7447 Pin 2
8 const int pinD = 5; // IC 7447 Pin 6
9
10 const int pinSegG = 12; // Segment G resistor line (Active-LOW bypass)

```

```

11
12 const int digitPins[6] = {6, 7, 8, 9, 10, 11}; // Digit enable pins
13
14 // Sound & Control Pins
15 const int pinBuzzer = A0; // Piezo Buzzer Output
16 const int btnMode = A1; // Mode Switch (Normal -> Set Clock -> Set Alarm)
17 const int btnMin = A2; // Increment Minutes
18 const int btnHrs = A3; // Increment Hours
19 const int btnAlarm = A4; // Alarm Toggle / Silence
20
21 // Debounce trackers
22 unsigned long lastDebounceMode = 0;
23 unsigned long lastDebounceMin = 0;
24 unsigned long lastDebounceHrs = 0;
25 unsigned long lastDebounceAlarm = 0;
26 const unsigned long debounceDelay = 200; // ms
27
28 // Active-LOW Lookup Table for Bypassed Segment G (LOW = ON, HIGH = OFF)
29 const bool segGState[10] = {HIGH, HIGH, LOW, LOW, LOW, LOW, LOW, HIGH, LOW, LOW};
30
31 // Real-Time Clock Digits (Starts at 12:00:00)
32 int secUnits = 0;
33 int secTens = 0;
34 int minUnits = 0;
35 int minTens = 0;
36 int hrsUnits = 2;
37 int hrsTens = 1;
38
39 // Alarm Target Digits (Default 12:01:00)
40 int alarmMinUnits = 1;
41 int alarmMinTens = 0;
42 int alarmHrsUnits = 2;
43 int alarmHrsTens = 1;
44
45 // System States
46 int currentMode = 0; // 0 = Normal Clock, 1 = Set Time, 2 = Set Alarm
47 bool alarmEnabled = false; // Alarm On/Off Toggle
48 bool alarmRinging = false; // Alarm Active Tone State
49
50 unsigned long previousMillis = 0;
51
52 void setup() {
53   pinMode(pinA, OUTPUT);
54   pinMode(pinB, OUTPUT);
55   pinMode(pinC, OUTPUT);
56   pinMode(pinD, OUTPUT);
57
58   pinMode(pinSegG, OUTPUT);
59   digitalWrite(pinSegG, HIGH);
60
61   for (int i = 0; i < 6; i++) {
62     pinMode(digitPins[i], OUTPUT);
63     digitalWrite(digitPins[i], LOW);
64   }
65
66   pinMode(pinBuzzer, OUTPUT);
67
68   // Configure push buttons with internal pull-ups (PRESS = LOW)
69   pinMode(btnMode, INPUT_PULLUP);
70   pinMode(btnMin, INPUT_PULLUP);
71   pinMode(btnHrs, INPUT_PULLUP);
72   pinMode(btnAlarm, INPUT_PULLUP);
73 }
74
75 // K-Map Minimized Next-State Function for BCD Modulo-10 Counter (0 to 9)
76 // Boolean Logic: Anext = !A | Bnext = B!A + !D!BA | Cnext = C!A + C!B + !CBA | Dnext =
77 // D!A + CBA
78 int kmapMod10Next(int val) {
79   bool A = val & 0x01;
80   bool B = (val >> 1) & 0x01;
81   bool C = (val >> 2) & 0x01;
82   bool D = (val >> 3) & 0x01;

```

```

83     bool Anext = !A;
84     bool Bnext = (B && !A) || (!D && !B && A);
85     bool Cnext = (C && !A) || (C && !B) || (!C && B && A);
86     bool Dnext = (D && !A) || (C && B && A);
87
88     return (Dnext << 3) | (Cnext << 2) | (Bnext << 1) | Anext;
89 }
90
91 // K-Map Minimized Next-State Function for Modulo-6 Counter (0 to 5)
92 // Boolean Logic: Xnext = !X | Ynext = Y!X + !Z!YX | Znext = Z!X + YX
93 int kmapMod6Next(int val) {
94     bool X = val & 0x01;
95     bool Y = (val >> 1) & 0x01;
96     bool Z = (val >> 2) & 0x01;
97
98     bool Xnext = !X;
99     bool Ynext = (Y && !X) || (!Z && !Y && X);
100    bool Znext = (Z && !X) || (Y && X);
101
102    return (Znext << 2) | (Ynext << 1) | Xnext;
103 }
104
105 // K-Map Minimized 12-Hour State Transition Engine (12 -> 01 transition)
106 void incrementHoursKMap(int &hTens, int &hUnits) {
107     if (hTens == 1 && hUnits == 2) {
108         hTens = 0;
109         hUnits = 1;
110     } else if (hTens == 0 && hUnits == 9) {
111         hTens = 1;
112         hUnits = 0;
113     } else {
114         hUnits = kmapMod10Next(hUnits);
115     }
116 }
117
118 // K-Map Minimized Minute Transition Engine (00 -> 59)
119 void incrementMinutesKMap(int &mTens, int &mUnits) {
120     if (mUnits == 9) {
121         mUnits = 0;
122         if (mTens == 5) {
123             mTens = 0;
124         } else {
125             mTens = kmapMod6Next(mTens);
126         }
127     } else {
128         mUnits = kmapMod10Next(mUnits);
129     }
130 }
131
132 // Time Advance Logic driven by K-Map Functions
133 void advanceTimeKMap() {
134     int nextSecUnits = kmapMod10Next(secUnits);
135
136     if (secUnits == 9) {
137         secUnits = 0;
138         int nextSecTens = kmapMod6Next(secTens);
139
140         if (secTens == 5) {
141             secTens = 0;
142
143             int prevMinUnits = minUnits;
144             int prevMinTens = minTens;
145             incrementMinutesKMap(minTens, minUnits);
146
147             if (prevMinTens == 5 && prevMinUnits == 9 && minUnits == 0 && minTens == 0) {
148                 incrementHoursKMap(hrsTens, hrsUnits);
149             }
150         } else {
151             secTens = nextSecTens;
152         }
153     } else {
154         secUnits = nextSecUnits;
155     }

```



```

156
157 // Trigger Alarm Match Check
158 if (alarmEnabled && hrsTens == alarmHrsTens && hrsUnits == alarmHrsUnits &&
159     minTens == alarmMinTens && minUnits == alarmMinUnits && secTens == 0 && secUnits
160     == 0) {
161     alarmRinging = true;
162 }
163
164 void beep(int duration) {
165     tone(pinBuzzer, 1000);
166     delay(duration);
167     noTone(pinBuzzer);
168 }
169
170 void handleButtons() {
171     unsigned long currentMillis = millis();
172
173     // Button 1: Mode Switch (A1)
174     if (digitalRead(btnMode) == LOW) {
175         if (currentMillis - lastDebounceMode > debounceDelay) {
176             currentMode = (currentMode + 1) % 3;
177             lastDebounceMode = currentMillis;
178             beep(100);
179         }
180     }
181
182     // Button 2: Minutes Increment (A2)
183     if (digitalRead(btnMin) == LOW) {
184         if (currentMillis - lastDebounceMin > debounceDelay) {
185             if (currentMode == 1) {
186                 incrementMinutesKMap(minTens, minUnits);
187                 secUnits = 0;
188                 secTens = 0;
189             } else if (currentMode == 2) {
190                 incrementMinutesKMap(alarmMinTens, alarmMinUnits);
191             }
192             lastDebounceMin = currentMillis;
193             beep(50);
194         }
195     }
196
197     // Button 3: Hours Increment (A3)
198     if (digitalRead(btnHrs) == LOW) {
199         if (currentMillis - lastDebounceHrs > debounceDelay) {
200             if (currentMode == 1) {
201                 incrementHoursKMap(hrsTens, hrsUnits);
202             } else if (currentMode == 2) {
203                 incrementHoursKMap(alarmHrsTens, alarmHrsUnits);
204             }
205             lastDebounceHrs = currentMillis;
206             beep(50);
207         }
208     }
209
210     // Button 4: Alarm Toggle / Stop (A4)
211     if (digitalRead(btnAlarm) == LOW) {
212         if (currentMillis - lastDebounceAlarm > debounceDelay) {
213             if (alarmRinging) {
214                 alarmRinging = false;
215                 noTone(pinBuzzer);
216             } else {
217                 alarmEnabled = !alarmEnabled;
218                 beep(100);
219                 if (alarmEnabled) {
220                     delay(100);
221                     beep(100);
222                 }
223             }
224             lastDebounceAlarm = currentMillis;
225         }
226     }
227 }

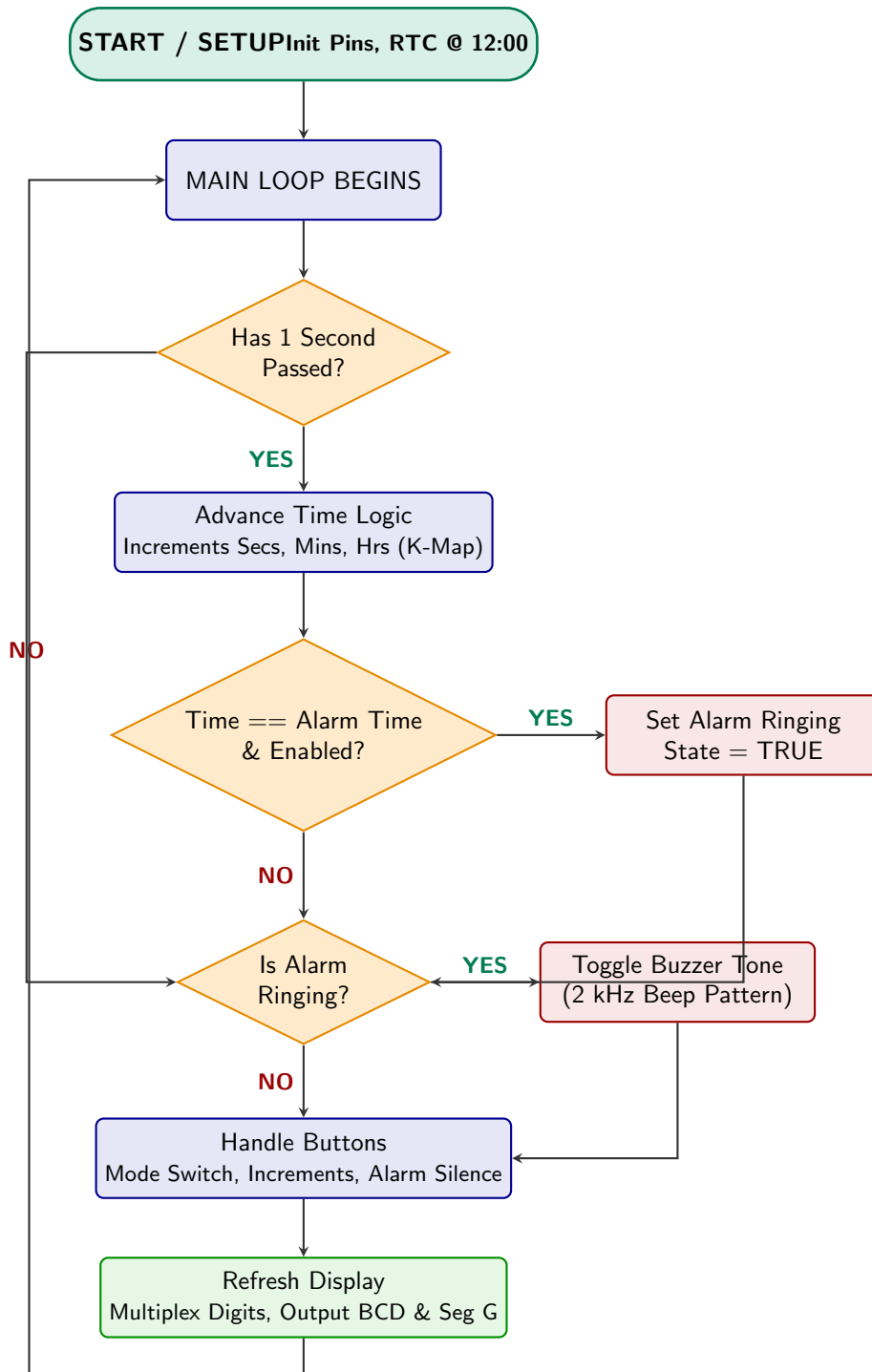
```

```

228
229 void loop() {
230     unsigned long currentMillis = millis();
231     if (currentMillis - previousMillis >= 1000) {
232         previousMillis += 1000;
233         advanceTimeKMap();
234     }
235
236     // Sound Buzzer Pattern when Alarm Triggers
237     if (alarmRinging) {
238         if ((millis() / 250) % 2 == 0) {
239             tone(pinBuzzer, 2000);
240         } else {
241             noTone(pinBuzzer);
242         }
243     }
244
245     handleButtons();
246     refreshDisplay();
247 }
248
249 void writeBCDAndBypasses(int val) {
250     // Output BCD value to 7447 IC pins (A, B, C, D)
251     digitalWrite(pinA, (val & 0x01) ? HIGH : LOW);
252     digitalWrite(pinB, (val & 0x02) ? HIGH : LOW);
253     digitalWrite(pinC, (val & 0x04) ? HIGH : LOW);
254     digitalWrite(pinD, (val & 0x08) ? HIGH : LOW);
255
256     // Output manual segment G line only
257     digitalWrite(pinSegG, segGState[val]);
258 }
259
260 void refreshDisplay() {
261     int digits[6];
262
263     if (currentMode == 0 || currentMode == 1) {
264         // Clock Mode: Display real-time clock (HH:MM:SS)
265         digits[0] = hrsTens;
266         digits[1] = hrsUnits;
267         digits[2] = minTens;
268         digits[3] = minUnits;
269         digits[4] = secTens;
270         digits[5] = secUnits;
271     } else if (currentMode == 2) {
272         // Alarm Mode: Display set alarm time (HH:MM:00)
273         digits[0] = alarmHrsTens;
274         digits[1] = alarmHrsUnits;
275         digits[2] = alarmMinTens;
276         digits[3] = alarmMinUnits;
277         digits[4] = 0;
278         digits[5] = 0;
279     }
280
281     for (int i = 0; i < 6; i++) {
282         for (int j = 0; j < 6; j++) {
283             digitalWrite(digitPins[j], LOW);
284         }
285
286         writeBCDAndBypasses(digits[i]);
287
288         digitalWrite(digitPins[i], HIGH);
289         delayMicroseconds(1000);
290         digitalWrite(digitPins[i], LOW);
291     }
292 }

```

Listing 1: 6-Digit Clock & Alarm System Core Engine



7 Initialization (1–77)

The initial setup section of the code handles global variable definitions, hardware pin allocations for the 7447 decoder, segment bypasses, and push buttons, along with configuring their pin modes and internal pull-ups.

8 Code Breakdown & Functionality

8.1 K-Map Minimized Modulo-10 Counter Logic

1. **Unpacking the Number:** For a K-map, a single decimal digit is represented using its individual BCD binary bits (D, C, B, A).
 - `val & 0x01` looks only at the very first bit (1s place).
 - `(val >> 1) & 0x01` shifts the number one spot to the right to look at the 2s place (bit B).
 - `>> 2` and `>> 3` shift to extract bit C (4s place) and bit D (8s place).
 - Example: If `val = 6` (binary 0110), then $D = 0, C = 1, B = 1, A = 0$.
2. **Solving the K-Map Equations:** Applies K-map minimized Boolean logic to calculate the next state:

$$B_{\text{next}} = B\bar{A} + \bar{D}\bar{B}A$$

3. **Packing It Back Together:** Combines individual bits ($A_{\text{next}}, B_{\text{next}}, C_{\text{next}}, D_{\text{next}}$) using bitwise shifts (`<<`) and bitwise OR (`|`) operators to return a single combined integer.

8.2 K-Map Minimized Modulo-6 Counter Logic

- **Bit Extraction:** Unpacks the current state value into individual bits (X, Y, Z) using bitwise shifts.
- **Next-State Evaluation:** Applies K-map minimized Boolean logic to compute the next states ($X_{\text{next}}, Y_{\text{next}}, Z_{\text{next}}$) for a modulo-6 counter.
- **Bit Recombination:** Shifts and combines the evaluated bits back into a single integer format before returning.

8.3 12-Hour and Minute Transition Engines (Lines 105–130)

- **Hour Transition Control:** Detects when the clock reaches 12 (`hTens == 1` and `hUnits == 2`) and wraps it back around to 01. Handles normal hour increments by shifting 09 to 10 or passing single digits through the modulo-10 K-map function.
- **Minute and Second Rollover:** Checks if the minute units digit has reached 9, resetting it to 0 and triggering the tens digit logic. Advances the minute tens digit using the modulo-6 K-map function, or resets it entirely to 0 if it reaches 5.

8.4 Time Advance Coordination (`advanceTimeKMap`, Lines 132–161)

- **Coordination Logic:** Computes the next second units value using the modulo-10 K-map function, resetting it to 0 and triggering the second tens digit via the modulo-6 K-map function whenever units reach 9. Propagates changes upward by checking if seconds have completed a full minute, which then triggers the minute and potentially the hour transition engines.
- **Alarm Trigger Verification:** Checks if the alarm is enabled and if the current clock time perfectly matches the target alarm time down to the hour, minute, and zeroed seconds. Sets the `alarmRinging` state to `true` if a complete match is found, activating the buzzer pattern.

8.5 Buzzer & Debounce Management (Lines 164–195)

- **Buzzer Function (`beep`):** Generates an audible tone at 1000 Hz on the designated buzzer pin for a specified duration, turning off the tone using `noTone` after the delay finishes.
- **Button and Debounce Management (`handleButtons`):** Tracks elapsed time via `millis()` to filter out physical button bounce noise based on a defined delay threshold. Cycles through system modes (Normal Clock, Set Time, Set Alarm) when the mode button is pressed, and increments minutes and hours for either the clock or alarm depending on the active mode.

8.6 Hours & Alarm Control (Lines 197–227)

- **Hours Button Control (btnHrs):** Checks if the hours increment button is pressed with proper software debouncing applied. Increments either the real-time clock hours or the alarm hours using the K-map engine based on the active system mode.
- **Minutes Button Control (btnMin) :** Checks if the minutes increment button is pressed with proper software debouncing applied. Increments either the real-time clock mins or the alarm mins using the K-map engine based on the active system mode.
- **Alarm Toggle and Silence Control (btnAlarm):** Silences an active ringing alarm and turns off the buzzer immediately when pressed. Toggles the alarm system state on or off if the alarm is not currently ringing, providing auditory confirmation beeps.

8.7 Main Loop & BCD Handler (Lines 228–258)

- **Main Loop Timing and Execution (loop):** Tracks time with `millis()` to call `advanceTimeKMap()` exactly once every 1000 milliseconds (1 second) to keep the clock accurate. Controls an intermittent beeping pattern at 2000 Hz when the alarm is actively ringing, and continuously calls button-handling and display-refresh functions.
- **BCD and Bypass Output Handler (writeBCDAndBypasses):** Extracts individual bits from the given value using bit masks and sets the hardware pins (`pinA` through `pinD`) HIGH or LOW to drive the 7447 BCD decoder IC. Writes the pre-calculated state to the manual segment G bypass line based on the segment lookup table.

8.8 Display Buffer & Multiplexing (Lines 260–292)

- **Display Buffer Allocation (refreshDisplay):** Populates a 6-digit array with current real-time clock values (*HH : MM : SS*) when the system is in normal or time-setting mode. Loads the target alarm time values (*HH : MM : 00*) into the array with zeroed seconds when the system is switched to alarm-setting mode.
- **Multiplexing and Refresh Logic:** Loops through all six digit pins, temporarily turning them all LOW to clear ghosting before writing the appropriate BCD and bypass data for the current digit. Pulses each individual digit pin HIGH for one millisecond sequentially, creating the persistence-of-vision effect required to display all six digits at once.

9 Project Media

9.1 Video Demonstration

- **YouTube Demonstration Link:** [Click Here to Watch Video on YouTube](#)

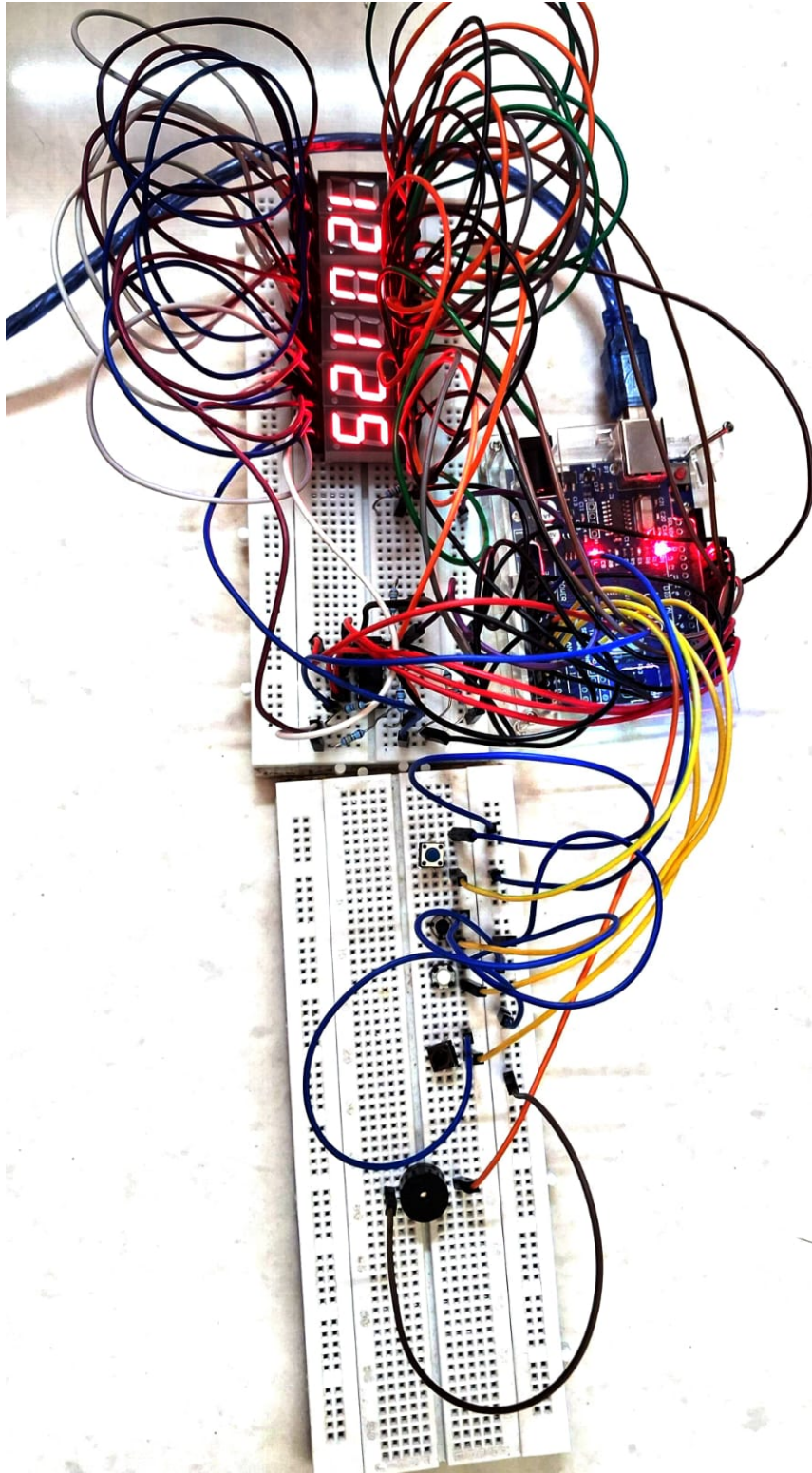
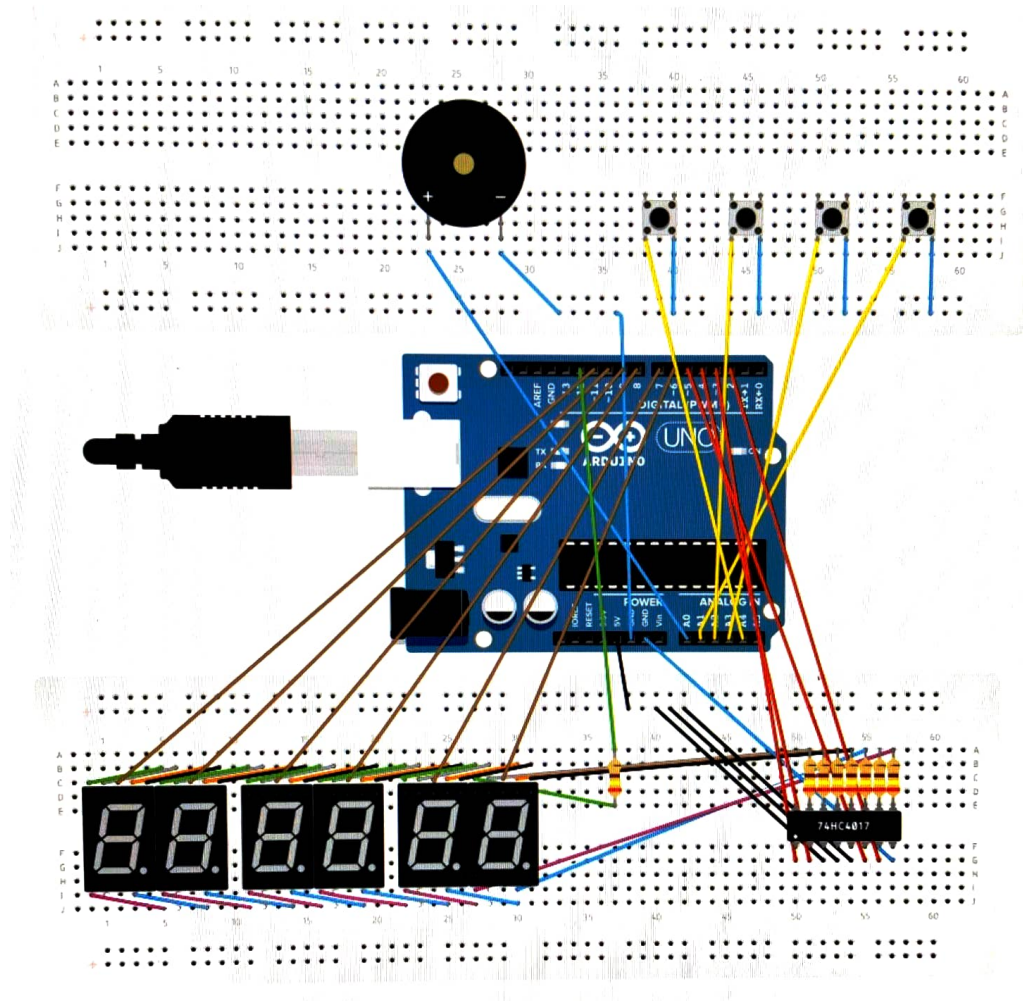


Figure 5: Final Hardware Setup of the 6-Digit Digital Clock and Alarm System.

9.2 Final Prototype Picture



10 Future Enhancements & Roadmap

Moving forward, our main goal is to clean up the build and make the clock much easier to use every day:

- **Cleaner Wiring:** We want to replace the messy jumper wires with custom-cut jumper wires that sit flat and neat directly on the breadboard. Cleaning up the wiring will give the clock a much tidier look and make the whole circuit more reliable.
- **Voice commands and clap activated alarm:** To make waking up less stressful, we plan to add a simple microphone feature so you can stop the alarm just by clapping or giving voice commands to set and off an alarm. This way, you won't have to hunt for tiny buttons in the dark when you're half-asleep.

11 Conclusion

This project successfully designed, simulated, and implemented a 6-digit digital clock and alarm system (*HH : MM : SS*) in a 12-hour format using an Arduino microcontroller and a 7447 BCD-to-7-segment decoder. By employing Karnaugh Map (K-Map) minimized Boolean logic equations for state transitions instead of traditional decimal arithmetic, the system demonstrates an efficient hardware-oriented algorithm for time progression and rollover logic. The integration of display multiplexing with a dedicated Segment-G bypass line produced a clear, flicker-free visual output, while the debounced four-button control matrix provided responsive user interface switching between normal display, time-setting, and alarm configuration modes. The functional physical prototype confirms the practical reliability of combining digital logic design principles with embedded software control.